

Engineering of Data Analysis

Lecture 2

Today's goal

1. Modern hardware is parallel — multiple cores, GPUs, clusters of machines.
2. To use this parallelism, we need programs where operations can run independently.
3. Many data analysis programs already have this property – they transform each row, or group of rows, independently.
4. This lecture introduce the **map/reduce** programming model that captures this structure and shows that **Pandas programs are already written in this model**.
5. Later lectures: **Spark** and **cuDF** take the same programming model and run it on parallel hardware.

Outline

Computing infrastructures

Programming data analysis – thinking parallel

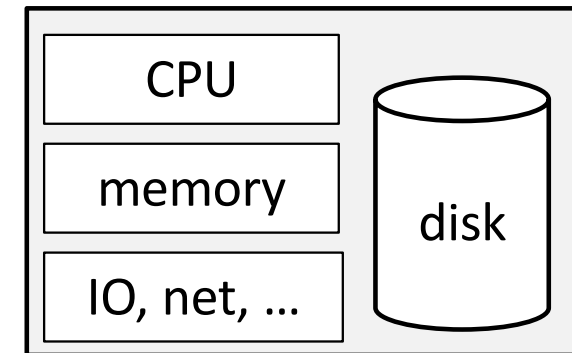
Pandas

- Dataframe programming
- Combining dataframes

Computer architecture

The key components of a computer are:

- CPU – for executing programs (composed by sequences of instructions). Modern CPUs have multiple cores, able to run several programs in parallel.
- Memory – for holding programs and data being processed.
- Disks – for storing programs and data persistently.
- IO devices – for interacting with users and other computers.

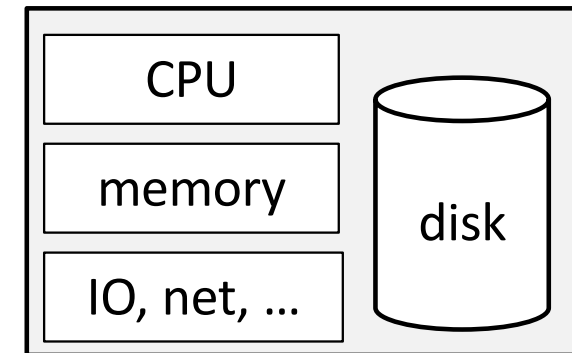


CPUs

The **central processing unit (CPU)** is the main processor of a computer and runs generic instructions. It is responsible for executing the code of programs.

A CPU runs one instruction at a time. The number of instruction executed per second depends on the CPU clock rate (and on the instruction).

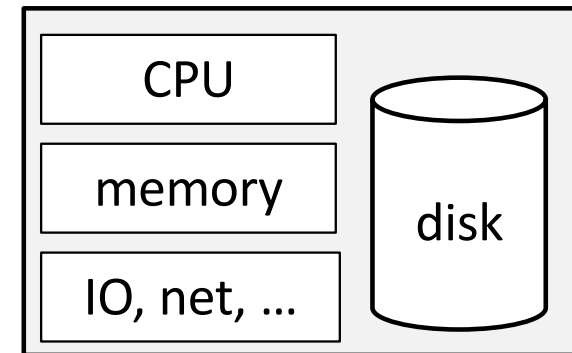
- This is not completely precise, but close.
- E.g. recent ARM processors can execute 2 to 4 multiplications per clock cycle. Running at 3 GHz, they can execute up to 12 ($4 * 3 * 10^9$) billion multiplications per second.



CPUs (2)

Modern processors include multiple CPUs (or cores).

- Currently, from 4 to 16 cores is common, even in mobile phones.

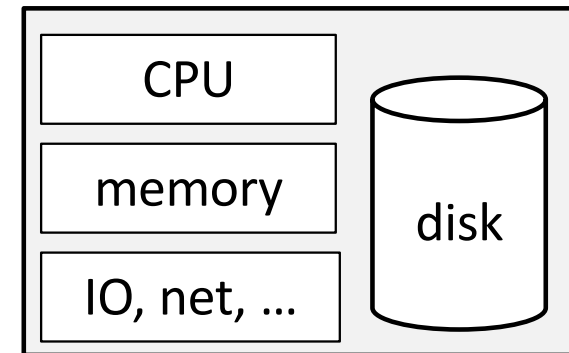


CPUs and Python

Python programs are interpreted, i.e., there is a program that reads the Python code and for each line executes the respective operation.

This leads to a high overhead – e.g. a multiplication can take 20 to 100 clock cycles.

- Note: there are some optimizations and some versions of Python that compile the code to machine code.



Example

```
[2] array = np.random.rand(10000000)
```

```
▶ %%timeit
```

```
total_sum_code = 0.0  
for number in array:  
    total_sum_code += number
```

```
⇒ 1.92 s ± 364 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)
```



```
%%timeit
```

```
total_sum_sum = sum(array)
```



```
972 ms ± 4.46 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)
```

```
[3] %%timeit
```

```
total_sum_lib = array.sum()
```

```
⇒ 21.4 ms ± 3.36 ms per loop (mean ± std. dev. of 7 runs, 10 loops each)
```

More examples in the notebook.

CPUs and Python

Why is using a numpy function 50-100x faster than using Python functions (or a program written fully in Python)?

Because numpy functions are optimized and compiled to machine code (from C). The same happens with other libraries of good quality – e.g. Pandas, scikit-learn.

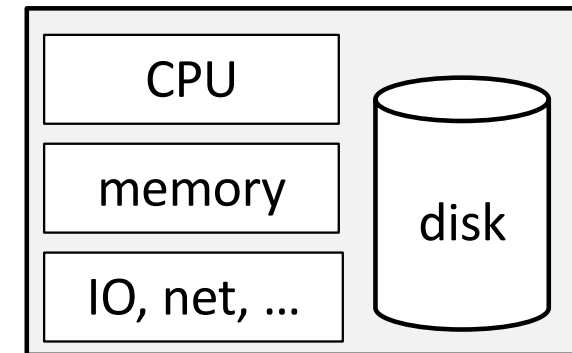
Key takeaway: use library functions whenever possible.

Memory, disk and caches

Data is stored on disks for persistency.

CPUs can only access data that is in memory, so data and programs needs to be moved from disk to memory before being used.

CPUs have special memory spaces, called caches (L1, L2 and L3 caches), for faster access to data. Data is automatically copied from memory to caches (data accessed is always in L1) and back.

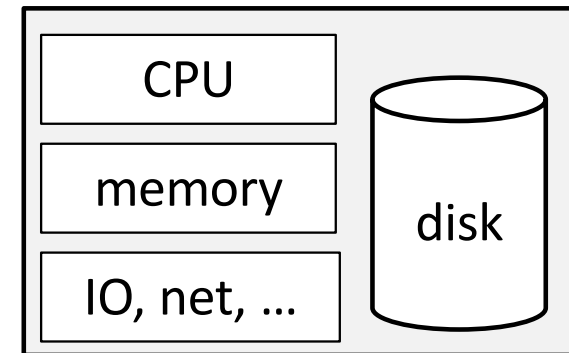


Memory, disk and caches

Time to access data:

- Caches (L1/L2/L3): 1–64 MB, 1–40 clock cycles (~ 1–40 ns)
- RAM: 16–128 GB, 60–100 clock cycles (~ 60–100 ns)
- SSD Disk: 1–8 TB, 10–100 μ s

For context: a 10-million-row DataFrame of 10 float columns $\approx$ 800 MB — fits in RAM, not in cache.



Example

Comparing the time it takes to sum 10^6 numbers that are close by in memory to far apart.

```
[15] arraydouble = [[i * 100 + j for j in range(10000)] for i in range(10000)]
```

```
▶ %%timeit  
  
sum_arraydouble = 0  
for i in range(1000):  
    for j in range(10000):  
        sum_arraydouble += arraydouble[i][j]
```

↗ 677 ms ± 14.9 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)

```
▶ %%timeit  
  
sum_arraydouble = 0  
for j in range(10000):  
    for i in range(1000):  
        sum_arraydouble += arraydouble[i][j]
```

↗ 1.45 s ± 256 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)

Memory and Python

Why is summing the same number of numbers that are closeby in memory faster than numbers that are far apart?

Because more data needs to be moved from RAM memory to the caches and back and because accessing data in caches is faster. (data is moved in blocks)

Key takeaway: improve data locality by using contiguous data structures instead of linked list; improve access locality... not very easy for programmers...

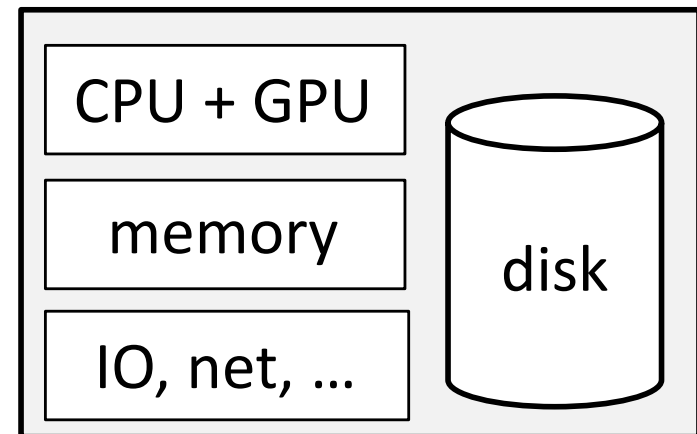
GPUs, TPUs and data analysis

A **graphics processing unit (GPU)** is a specialized processor designed to accelerate the creation of images.

- Base operations are floating-point operations, but not only.

Modern GPUs may include thousands of cores, with each core executing the same operation at the same time on different data.

- This is not completely precise, but close.



GPUs, TPUs and data analysis (2)

A **tensor processing unit (TPU)** is a specialized processor designed by Google to accelerate AI processing, specifically neural networks machine learning.

- It is similar to a GPU, but optimized to do matrix operations (which are the base of neural networks).

Recent TPUs, have tens of thousands of units that execute operations in parallel.



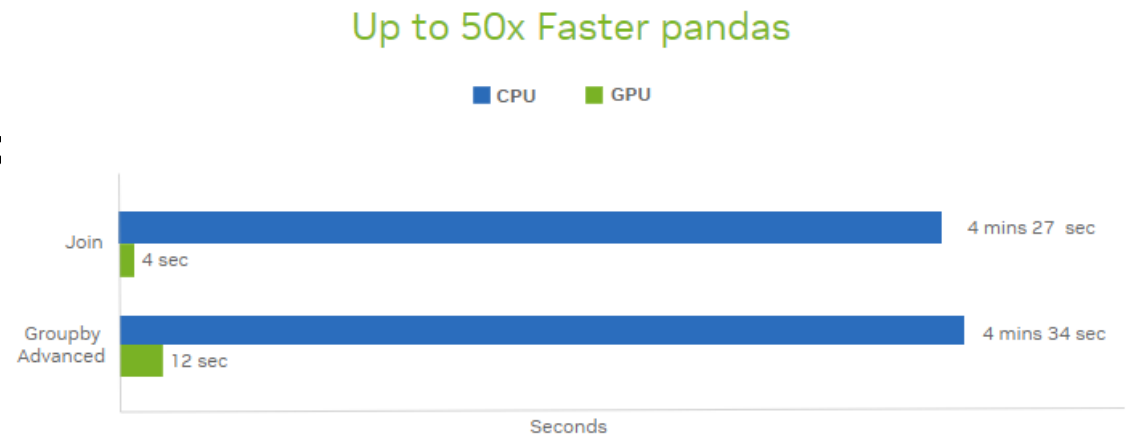
Source: <https://cloud.google.com/tpu> (accessed: Sep, 2020)

Data analysis on GPUs

Many data-analysis tasks consist in applying the same code to different data.

Several data processing framework exploit GPUs for improved performance.

E.g. cuDF is Pandas accelerated using GPUs.



Source: <https://developer.nvidia.com/blog/rapids-cudf-instantly-accelerates-pandas-up-to-50x-on-google-colab/> (accessed: Feb, 2025)

Data analysis on a single computer

The capacity of a single computer is constrained by:

- Disk space (for storing input and outputs)
- Memory (for holding data being processed)
- Processing units (for processing data)

A single computer is not enough to perform some data analysis jobs!

Google example: **generate web index**

20+ billion web pages x 20KB = 400+ TB

- ~1,000 hard drives to store the web

1 computer reads 30-35 MB/sec from disk

- ~4 months to read the web

Not even accounting for the computation itself...

Old school large-scale computing infrastructure

In the past, it was common for big companies (telecom, energy, etc.) to have “big machines” (super-computers, mainframes)

- Very expensive
- Some still do, but infrastructures have been evolving...



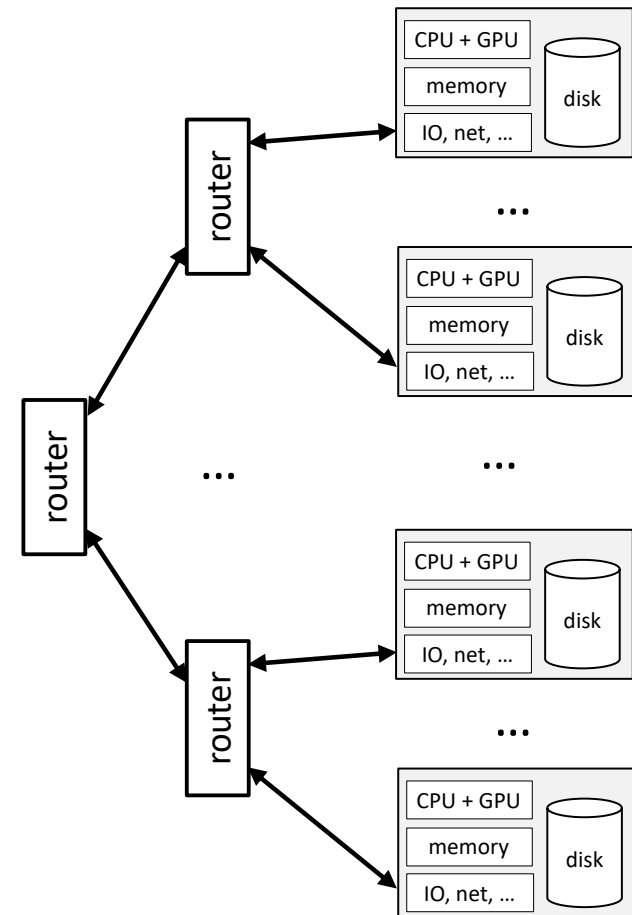
New age large-scale computing infrastructure

Most common infrastructure composed by:

- Cluster of commodity (Linux) machines
- Commodity network (ethernet) to connect them
- Cloud platforms have data center with 50K-100K machines

Advantages

- **Cheaper**: no special hardware
- Easier to **scale**: just add more machines
- Easier to handle **faults**: just replace machine by a new one



Computing infrastructure



Source: <https://www.google.com/about/datacenters/gallery/>

Data processing: a word of caution

Many analysis jobs are of “PC size” data and could be performed on a desktop PC, notebook or small server.

2TB SSD drives are already affordable and fast, offer high bandwidth, even with random access patterns

+

16 core CPUs + modern GPUs

=

can crunch a lot of data in little time.

GPUs are useful for many data processing jobs and many frameworks exist that explore GPUs.

Data processing: a word of caution (2)

Even when data processing can be performed in a single machine, it might be useful to use a shared server or a cloud machine. For example, for:

- Sharing expensive hardware (GPUs) among multiple users;
- Simplify accessing shared data.

Some alternatives are: Google Colab, Amazon Sagemaker, VS code, Databricks, etc.

The implication for programming

We have seen that modern computing infrastructure is parallel at every level -- from multi-core CPUs to GPU arrays to clusters of machines.

But **parallelism is only useful if programs can exploit it**. This requires operations that are **independent** – so they can be split across cores or machines without coordination.

Not all programs can be written this way. But **data transformations often can** – because many operations process each row, or each group of rows, independently.

Outline

Computing infrastructures

Programming data analysis – thinking parallel

Pandas

- Dataframe programming
- Combining dataframes

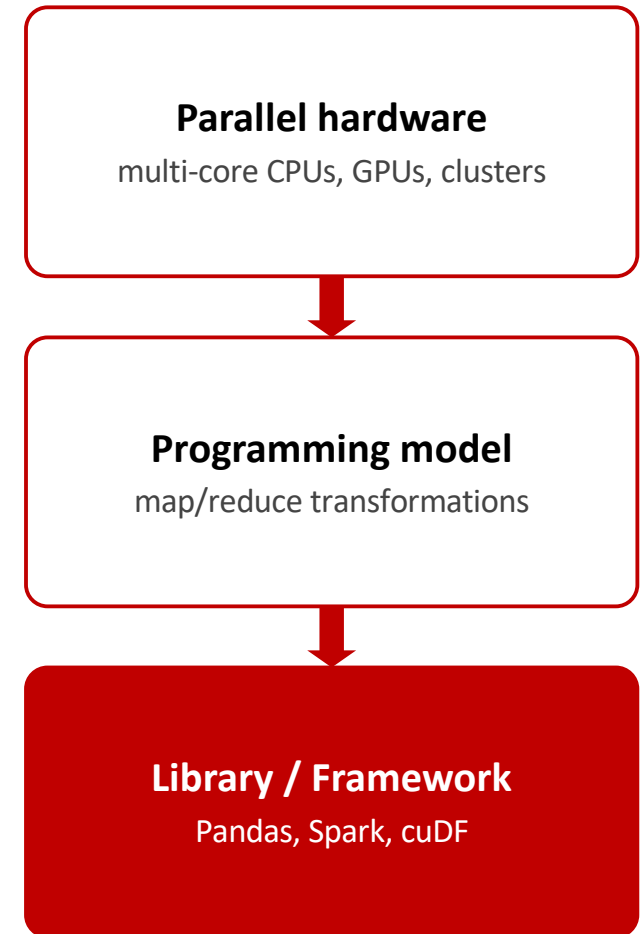
Programming and the computing infrastructure

The **map/reduce** model is a programming paradigm designed for parallel execution:

- **Map** transformations process each row independently – trivially parallel.
- **Reduce** transformations aggregate rows – partially parallel when commutative and associative.

Structure your computation as a sequence of map and reduce transformations, and it is possible to **explore parallelism** under the hood.

This is the model we introduce next.



Programming data analysis

Many data analysis computation are performed over tabular data.

Tabular data is data that can be represented in tables, with an instance per row, each instance with multiple attributes.

We will call this tabular data table or DataFrame.

Name	Age	Educational level	Company
Andrew	55	1	Good
Bernhard	43	2	Good
Carolina	37	5	Bad
Dennis	82	3	Good
Eve	23	3.2	Bad
Fred	46	5	Good
Gwyneth	38	4.2	Bad
Hayden	50	4	Bad
Irene	29	4.5	Bad
James	42	4.1	Good
Kevin	35	4.5	Bad
Lea	38	2.5	Good
Marcus	31	4.8	Bad
Nigel	71	2.3	Good

Programming data analysis (cont.)

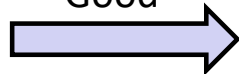
Data analysis programs are often defined as:

- a sequence of transformations;
- each transformation, transforms one DataFrame into another DataFrame.

Using this paradigm, how do you compute the *youngest person that is a good company?*

Name	Age	Educational level	Company
Andrew	55	1	Good
Bernhard	43	2	Good
Carolina	37	5	Bad
Dennis	82	3	Good
Eve	23	3.2	Bad
Fred	46	5	Good
Gwyneth	38	4.2	Bad
Hayden	50	4	Bad
Irene	29	4.5	Bad
James	42	4.1	Good
Kevin	35	4.5	Bad
Lea	38	2.5	Good
Marcus	31	4.8	Bad
Nigel	71	2.3	Good

Company
equals to
Good



Name	Age	Educational level	Company
Andrew	55	1	Good
Bernhard	43	2	Good
Dennis	82	3	Good
Fred	46	5	Good
James	42	4.1	Good
Lea	38	2.5	Good
Nigel	71	2.3	Good

Minimum
Age



Name	Age	Educational level	Company
Lea	38	2.5	Good

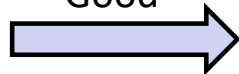
Programming data analysis (cont.)

Transformations can be:

- One-to-one transformations;
- Many-to-one transformations.

Name	Age	Educational level	Company
Andrew	55	1	Good
Bernhard	43	2	Good
Carolina	37	5	Bad
Dennis	82	3	Good
Eve	23	3.2	Bad
Fred	46	5	Good
Gwyneth	38	4.2	Bad
Hayden	50	4	Bad
Irene	29	4.5	Bad
James	42	4.1	Good
Kevin	35	4.5	Bad
Lea	38	2.5	Good
Marcus	31	4.8	Bad
Nigel	71	2.3	Good

Company
equals to
Good



Name	Age	Educational level	Company
Andrew	55	1	Good
Bernhard	43	2	Good
Dennis	82	3	Good
Fred	46	5	Good
James	42	4.1	Good
Lea	38	2.5	Good
Nigel	71	2.3	Good

Minimum
Age



Name	Age	Educational level	Company
Lea	38	2.5	Good

One-to-one (one-to-*) transformations

These transformations are also called **map** transformations.

A **map** transformation transforms each row of the Dataframe into another row of the Dataframe.

- E.g. adding a new column, removing a column, executing a computation on the value of a column, etc.

Why does this matter?

Because each row can be processed on a different core or machine — map transformations are **trivially parallel** (i.e. they can be executed in parallel without any special algorithm.)

This is the key property that makes the map/reduce model useful for exploiting parallel hardware.

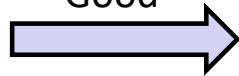
One-to-one (one-to-*) transformations (cont.)

More generally, the transformation can return zero, one or more rows.

When the function can return zero rows it is often called a **filter**.

Name	Age	Educational level	Company
Andrew	55	1	Good
Bernhard	43	2	Good
Carolina	37	5	Bad
Dennis	82	3	Good
Eve	23	3.2	Bad
Fred	46	5	Good
Gwyneth	38	4.2	Bad
Hayden	50	4	Bad
Irene	29	4.5	Bad
James	42	4.1	Good
Kevin	35	4.5	Bad
Lea	38	2.5	Good
Marcus	31	4.8	Bad
Nigel	71	2.3	Good

Company
equals to
Good



Name	Age	Educational level	Company
Andrew	55	1	Good
Bernhard	43	2	Good
Dennis	82	3	Good
Fred	46	5	Good
James	42	4.1	Good
Lea	38	2.5	Good
Nigel	71	2.3	Good

Many-to-one transformations

These transformations are also called **reduce** or **aggregation** transformations.

A **reduce** transformation transforms multiple rows of the Dataframe into a single row of the new Dataframe.

- E.g. statistical calculations over the values of a column, etc.

This transformation needs to get the data for all rows to be reduced.

Name	Age	Educational level	Company
Andrew	55	1	Good
Bernhard	43	2	Good
Dennis	82	3	Good
Fred	46	5	Good
James	42	4.1	Good
Lea	38	2.5	Good
Nigel	71	2.3	Good

Minimum
Age



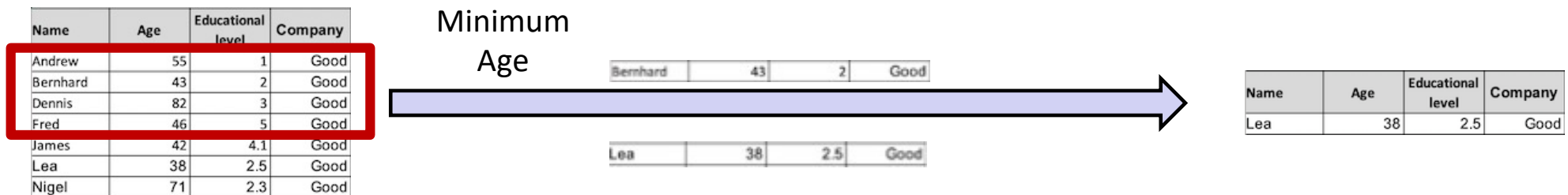
Name	Age	Educational level	Company
Lea	38	2.5	Good

Many-to-one transformations

Many transformations are (or can be made) commutative and associative. They can be performed partially independently and concurrently.

- E.g. min, max, avg, histogram, etc.

This is how a 16-core machine or a 100-node cluster can compute a global minimum without processing all rows sequentially — each core computes a local minimum, then the local results are combined.



Multiple many-to-one transformations

Often, we want to apply the transformation to different groups of rows at the same time. This can be done by grouping the rows of a Dataframe based on the value for some column.

In the previous example, we might want to know the youngest person for Good and for Bad company sub-groups.

Name	Age	Educational level	Company
Andrew	55	1	Good
Bernhard	43	2	Good
Carolina	37	5	Bad
Dennis	82	3	Good
Eve	23	3.2	Bad
Fred	46	5	Good
Gwyneth	38	4.2	Bad
Hayden	50	4	Bad
Irene	29	4.5	Bad
James	42	4.1	Good
Kevin	35	4.5	Bad
Lea	38	2.5	Good
Marcus	31	4.8	Bad
Nigel	71	2.3	Good

Group by
Company



Name	Age	Educational level	Company
Andrew	55	1	Good
Bernhard	43	2	Good
Dennis	82	3	Good
Fred	46	5	Good
James	42	4.1	Good
Lea	38	2.5	Good
Nigel	71	2.3	Good
Carolina	37	5	Bad
Eve	23	3.2	Bad
Gwyneth	38	4.2	Bad
Hayden	50	4	Bad
Irene	29	4.5	Bad
Kevin	35	4.5	Bad
Marcus	31	4.8	Bad

Minimum
Age



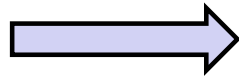
Company	Age
Good	38
Bad	23

Other examples

Compute the average age of good and bad companies?

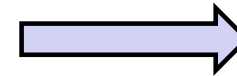
Name	Age	Educational level	Company
Andrew	55	1	Good
Bernhard	43	2	Good
Carolina	37	5	Bad
Dennis	82	3	Good
Eve	23	3.2	Bad
Fred	46	5	Good
Gwyneth	38	4.2	Bad
Hayden	50	4	Bad
Irene	29	4.5	Bad
James	42	4.1	Good
Kevin	35	4.5	Bad
Lea	38	2.5	Good
Marcus	31	4.8	Bad
Nigel	71	2.3	Good

Group by
Company



Name	Age	Educational level	Company
Andrew	55	1	Good
Bernhard	43	2	Good
Dennis	82	3	Good
Fred	46	5	Good
James	42	4.1	Good
Lea	38	2.5	Good
Nigel	71	2.3	Good
Carolina	37	5	Bad
Eve	23	3.2	Bad
Gwyneth	38	4.2	Bad
Hayden	50	4	Bad
Irene	29	4.5	Bad
Kevin	35	4.5	Bad
Marcus	31	4.8	Bad

Average
Age



Company	Age
Good	53.8
Bad	34.7

Other examples

Which group has lower average Educational Level: good or bad companies ?

Name	Age	Educational level	Company
Andrew	55	1	Good
Bernhard	43	2	Good
Carolina	37	5	Bad
Dennis	82	3	Good
Eve	23	3.2	Bad
Fred	46	5	Good
Gwyneth	38	4.2	Bad
Hayden	50	4	Bad
Irene	29	4.5	Bad
James	42	4.1	Good
Kevin	35	4.5	Bad
Lea	38	2.5	Good
Marcus	31	4.8	Bad
Nigel	71	2.3	Good

Group by
Company



Name	Age	Educational level	Company
Andrew	55	1	Good
Bernhard	43	2	Good
Dennis	82	3	Good
Fred	46	5	Good
James	42	4.1	Good
Lea	38	2.5	Good
Nigel	71	2.3	Good
Carolina	37	5	Bad
Eve	23	3.2	Bad
Gwyneth	38	4.2	Bad
Hayden	50	4	Bad
Irene	29	4.5	Bad
Kevin	35	4.5	Bad
Marcus	31	4.8	Bad

Average
Ed. Level



Company	Educational level
Good	2.84
Bad	4.31

Minimum
index



Good

Outline

Computing infrastructures

Programming data analysis – thinking parallel

Pandas

- **Dataframe programming**
- Combining dataframes

Pandas

Pandas is a “fast, powerful, flexible and easy to use open source data analysis and manipulation tool”

<https://pandas.pydata.org/>

Pandas adopts this functional paradigm, where programs are specified by transforming DataFrames into other DataFrames.

Pandas runs these operations sequentially on a single CPU. The next slides show Pandas operations and classify each as map or reduce.

DataFrame

In Pandas, a table is represented as a DataFrame object.

```
population = pd.DataFrame( { "country": ["PT", "ES", "DE"] , \
                             "population": [10276617, 46937060, 83019213]})

print( population)
```

	country	population
0	PT	10276617
1	ES	46937060
2	DE	83019213

DataFrame: load from a CSV file

`pd.read_csv( filename)`

Loads a CSV file into a DataFrame.

```
import os

# Let's create a PATH in a OS independent way
# File lec1-example.csv is in directory data
fileName = os.path.join( "data", "lec1-example.csv")

# Read a CSV file into a DataFrame
df = pd.read_csv(fileName)

print( df)
```

	Name	Age	Educational level	Company
0	Andrew	55	1.0	Good
1	Bernhard	43	2.0	Good
2	Carolina	37	5.0	Bad
3	Dennis	82	3.0	Good
4	Eve	23	3.2	Bad
5	Fred	46	5.0	Good
6	Gwyneth	38	4.2	Bad
7	Hayden	50	4.0	Bad
8	Irene	29	4.5	Bad
9	James	42	4.1	Good
10	Kevin	35	4.5	Bad
11	Lea	38	2.5	Good
12	Marcus	31	4.8	Bad
13	Nigel	71	2.3	Good

Selecting rows based on conditions

`dataframe[condition]`

Selects a subset of the rows based on a condition over the value of the columns.

For combining multiple conditions use & (and) or | (or).

E.g.: `df[(df["Company"]=="Good") & (df["Educational level"]>=3.0)]`

Map (filter) transformation
each row can be tested independently

```
# Select the persons that are good company.  
good = df[df["Company"]=="Good"]  
  
print(good)
```

	Name	Age	Educational level	Company
0	Andrew	55	1.0	Good
1	Bernhard	43	2.0	Good
3	Dennis	82	3.0	Good
5	Fred	46	5.0	Good
9	James	42	4.1	Good
11	Lea	38	2.5	Good
13	Nigel	71	2.3	Good

Selecting columns

`dataframe[[cols]]`

Selects a subset of the columns.

Map transformation
each row can be processed independently

```
person_age = df[["Name", "Age"]]  
  
print(person_age)
```

	Name	Age
0	Andrew	55
1	Bernhard	43
2	Carolina	37
3	Dennis	82
4	Eve	23
5	Fred	46
6	Gwyneth	38
7	Hayden	50
8	Irene	29
9	James	42
10	Kevin	35
11	Lea	38
12	Marcus	31
13	Nigel	71

Applying aggregation function

`dataframe[col].function()`

Applies function over the values of the selected column.

Some functions:

- min
- max
- mean
- median
- std
- ...

Reduce transformation
many rows aggregated into one value

```
minAge = good["Age"].min()  
print( "Minimum age is "  
print( minAge)
```

```
Minimum age is  
38
```

Series

Besides DataFrames, Pandas supports Series data type.

A Series is a sequence of values with an index (each value has an index) - it can be seen as a column of a DataFrame.

Aggregation functions are defined for Series.

```
print("Good dataframe")
print(good)
print("Name series")
print(good["Name"])
```

Good dataframe

	Name	Age	Educational level	Company
0	Andrew	55	1.0	Good
1	Bernhard	43	2.0	Good
3	Dennis	82	3.0	Good
5	Fred	46	5.0	Good
9	James	42	4.1	Good
11	Lea	38	2.5	Good
13	Nigel	71	2.3	Good

Name series

0	Andrew
1	Bernhard
3	Dennis
5	Fred
9	James
11	Lea
13	Nigel

Name: Name, dtype: object

Series (cont.)

`series.to_frame()`

Converts a Series into a Dataframe.

Map transformation
each row can be processed independently

```
good["Name"].to_frame()
```

	Name
0	Andrew
1	Bernhard
3	Dennis
5	Fred
9	James
11	Lea
13	Nigel

Applying aggregation function

`dataframe[[cols]].function()`

Applies function to multiple columns.

Reduce transformations
Each column aggregated independently

```
mins = good[["Age", "Educational level"]].min()  
print( "Minimum information for several columns now")  
print( mins)
```

```
Minimum information for several columns now  
Age                38.0  
Educational level   1.0  
dtype: float64
```

Applying aggregation function (cont.)

`dataframe.agg (spec)`

Allows to apply different aggregations function at once, possible to different columns.

Parallel Reduce transformations
many rows aggregated into one value per function – can be parallel across functions

```
stats = good.agg({'Age' : 'min', 'Educational level' : 'max'})  
print( "Minimum and max in the same operation")  
print( stats)
```

```
Minimum and max in the same operation  
Age                38.0  
Educational level   5.0  
dtype: float64
```

Applying aggregation function (cont.)

Some more functions:

- `nsmallest( count, [cols])`
- ...

Reduce transformation
many rows aggregated into N values

```
youngestGood = good.nsmallest(1,["Age"])  
print( good.nsmallest(1,["Age"]))
```

	Name	Age	Educational level	Company
11	Lea	38	2.5	Good

Grouping and applying aggregation function per group

`dataframe.groupby([col]).function()`

Parallel Reduce transformation
Each group is reduced independently — can be parallel across groups

groupby groups the rows of the DataFrame by the value of the specified column. For each group it is possible to apply a reduce/aggregation function.

```
youngest = df[["Age", "Company"]].groupby(["Company"]).min()  
print( youngest)
```

	Age
Company	
Bad	23
Good	38

Applying a custom aggregate function

`dataframe.groupby([col]).apply( function)`

Parallel Reduce transformation

The most general reduce pattern. The framework runs it per group but cannot optimise the function itself.

In some case, you might want to apply a custom aggregate function. In this case, it is possible to define the function and use the apply method.

```
def mysmallest( p):  
    return p.nsmallest(1, ["Age"])  
  
stats = df.groupby("Company").apply(mysmallest, include_groups=False)  
print( stats)
```

		Name	Age	Educational level
Company				
Bad	4	Eve	23	3.2
Good	11	Lea	38	2.5

Applying a custom aggregate function

`dataframe.groupby([col]).apply( function)`

In some case, you might want to apply a custom aggregate function. In this case, it is possible to define the function and use the apply method.

Using lambda functions:

```
[ ] stats = df.groupby("Company").apply(lambda p: p.nsmallest(1, ["Age"]), include_groups=False)
      print( stats)
```



		Name	Age	Educational level
Company				
Bad	4	Eve	23	3.2
Good	11	Lea	38	2.5

Summary: Pandas operations as map and reduce

Pandas operation	Transformation	Parallelism
df[condition] — filter rows	Map — each row tested independently	Trivially parallel
df[['col1','col2']] — select columns	Map — each row projected independently	Trivially parallel
df['col'].min() — aggregate	Reduce — all rows → one value	Parallel if commutative/associative
df.groupby('col').min() — group + aggregate	Reduce per group — groups independent	Parallel across groups
df.groupby('col').apply(f) — custom	Reduce per group — custom function	Parallel across groups, not within

Pandas runs all of these sequentially on one CPU.

Spark and cuDF take the same operations and distribute them across cores, GPUs, or machines. The programming model stays the same — the execution engine changes.

Outline

Computing infrastructures

Programming data analysis – thinking parallel

Pandas

- Dataframe programming
- **Combining dataframes**

Combining DataFrames

Sometimes data to be processed is not in a single table / DataFrame.

In this case, it is necessary to be able to combine multiple DataFrames. Two of the most useful operations are:

concat – combines DataFrames by taking rows from each of the DataFrames.

join – combines DataFrames by taking columns from each of the DataFrames.

Combining DataFrames

Sometimes data to be processed is not in a single table / DataFrame. Two situations:

- **Same columns, different rows** -- e.g. monthly sales files that need to be stacked.

Use **concat** – appends rows from one DataFrame to another.

- **Different columns, shared key** – e.g. a population table and a language table linked by country code.

Use **join** / **merge** -- combines columns from both DataFrames by matching rows on a key.

Join type determines what happens with unmatched rows.

Concat

pd.concat([dataframe, dataframe2])

Creates a new DataFrame that combines the lines of both DataFrames.

When a column does not exist in one of the dataframes, the value of the columns for the rows will be NaN.

columns for the rows will be NaN.

country	population
PT	10276617
ES	46937060
DE	83019213

+

capital	population	country
Brasilia	211049519	BR
Mexico City	127575529	MX
Montevideu	3461731	UY

→

country	population	capital
PT	10276617	NaN
ES	46937060	NaN
DE	83019213	NaN
BR	211049519	Brasilia
MX	127575529	Mexico City
UY	3461731	Montevideu

Concat: example

```
population1 = pd.DataFrame( { "country": ["PT", "ES", "DE"] , \
                               "population": [10276617, 46937060, 83019213]})

population2 = pd.DataFrame( {"capital" : ["Brasilia", "Mexico City", "Montevideu"],\
                              "population": [211049519, 127575529, 3461731], \
                              "country": ["BR", "MX", "UY"]})

population = pd.concat([population1,population2])
print( population)
```

	country	population	capital
0	PT	10276617	NaN
1	ES	46937060	NaN
2	DE	83019213	NaN
0	BR	211049519	Brasilia
1	MX	127575529	Mexico City
2	UY	3461731	Montevideu

Join

Creates a new DataFrame that combines the columns of both DataFrames.



Join on Pandas

```
dataframe.join(dataframe2,on=column,how="left"|"right"|"inner"|"outer")
```

The link between the two dataframes is done by:

- the column specified by **on** in dataframe;
- the index of dataframe2.

Join on Pandas (cont.)

```
population1 = pd.DataFrame( { "country": ["PT", "ES", "DE"]  
                             "population": [10276617, 46937060, 83019213]  
                           )  
  
print( population1)
```

	country	population
0	PT	10276617
1	ES	46937060
2	DE	83019213

Join on Pandas (cont.)

```
language = pd.DataFrame( { "country": [ "PT", "ES", "MX", "AR", "DE", "BR" ],  
                           "language": [ "Portuguese", "Spanish", "Spanish", "Spanish", "German", "Portuguese" ]  
                           )  
  
print( language )
```

	country	language
0	PT	Portuguese
1	ES	Spanish
2	MX	Spanish
3	AR	Spanish
4	DE	German
5	BR	Portuguese

Join on Pandas (cont.)

country	population
PT	10276617
ES	46937060
DE	83019213



country	language
DE	German
PT	Portuguese
ES	Spanish



country	population	language
PT	10276617	Portuguese
ES	46937060	Spanish
DE	83019213	German

```
countries1 = population1.join(language.set_index("country"), on="country")  
print(countries1)
```

	country	population	language
0	PT	10276617	Portuguese
1	ES	46937060	Spanish
2	DE	83019213	German

Join on Pandas

```
dataframe.join(dataframe2,on=column,how="left"|"right"|"inner"|"outer")
```

The link between the two dataframes is done by:

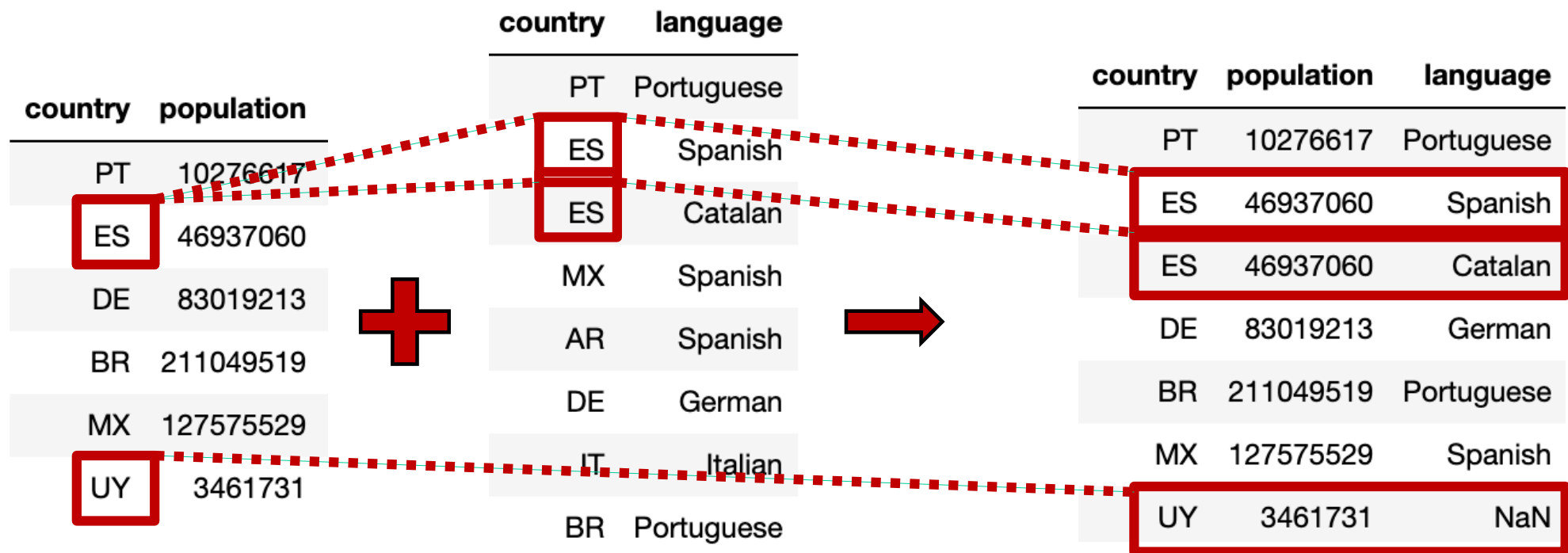
- the column specified by **on** in dataframe (or index if on is not specified);
- the index of dataframe2.

Alternative:

```
dataframe.merge(dataframe2,left_on=column,right_on=column,  
how="left"|"right"|"inner"|"outer")
```

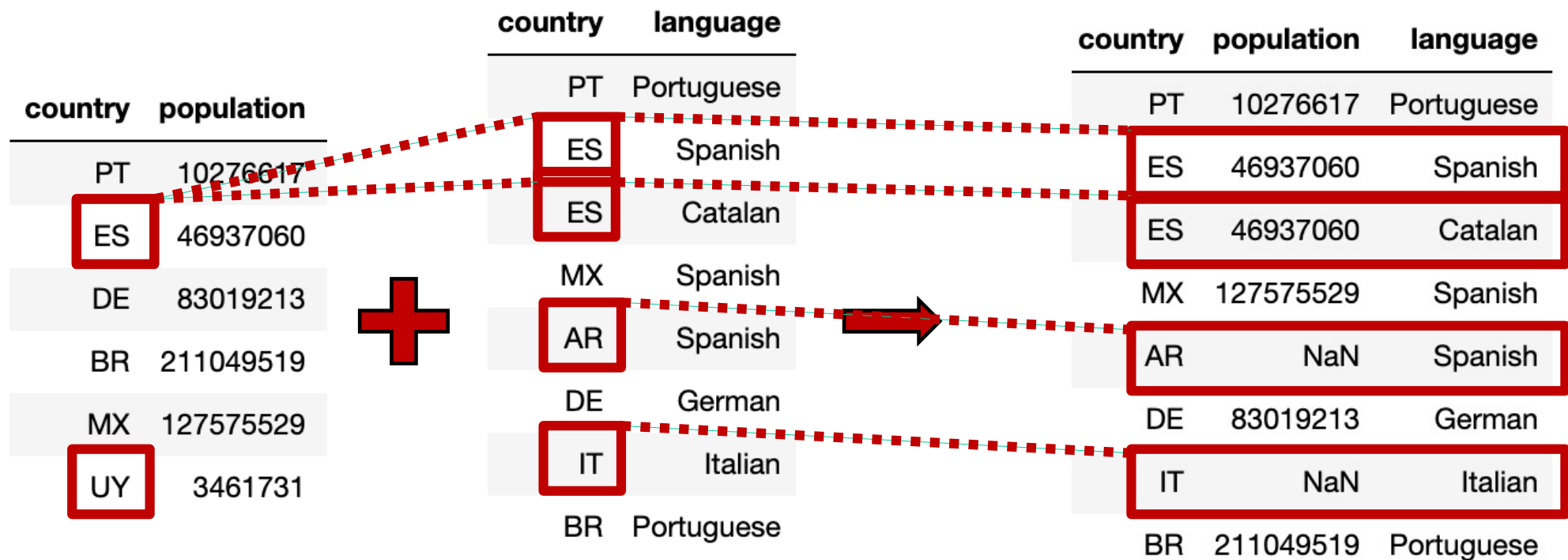
Left join (default mode)

For every row of the first table, join with all possible values of the second table. If no matching value exists in the second table, value will be NaN.



Right join

For every row of the second table, join with all possible values of the first table. If no matching value exists in the first table, value will be NaN.



Inner join

For every row of the first table, join with all possible values of the second table. If no matching value exists in the second table, the row will not appear.



Outer join

For every row of one table, join with all possible values of the other table. If no matching value exists in the other table, the row will have those columns has NaN.

country	population		country	language		country	population	language
PT	10276617		PT	Portuguese		PT	10276617	Portuguese
ES	46937060		ES	Spanish		ES	46937060	Spanish
DE	83019213		ES	Catalan		ES	46937060	Catalan
BR	211049519		MX	Spanish		DE	83019213	German
MX	127575529		AR	Spanish		BR	211049519	Portuguese
UY	3461731		DE	German		MX	127575529	Spanish
			IT	Italian		UY	3461731	NaN
			BR	Portuguese		AR	NaN	Spanish
						IT	NaN	Italian

Summary: from hardware to programming model

Map transformations are independent → can be parallelised across cores or machines.

Reduce transformations can be partially parallelised when commutative and associative.

Pandas programs are sequences of maps and reduces — but Pandas executes them sequentially on a single CPU.

Spark and cuDF execute the same model on many cores, GPUs, or many machines.

The programming model stays the same — the execution engine changes.

Next lectures: how Spark and cuDF work, and how to use them.

References